

UNIT-3

PUSHDOWN AUTOMATA AND PARSING ALGORITHMS: pushdown automata and contextfree languages;top down parsing and bottomup parsing;properties of cfl;application of pumping lemma,closure properties of cfl and decision algorithms.

PUSHDOWN AUTOMATA:

APPLICATION:

Regular languages are those which are accepted by finite automata.but a language such as $L = \{a^n b^n / n \geq 1\}$ cannot be accepted by a regular language because it has to precisely remember the number of a's and b's which would require infinite number of states.hence push down automata has been designed to accept context free languages.this is done by adding auxillary memory in the form of stack.

DEFINITIONS:

The pushdown automata formly consist of 7 tuples given by $M = (q, \Sigma, \Gamma, q_0, z_0, f)$

Where,

q :a finite non empty set of states

Σ :a finite non empty set of input symbols

δ :the transition function from $q \times (\Sigma^u \cup \Gamma^*)$ to the set of finite subsets $q \times \Gamma^*$

Γ :finite non empty set of push down symbols

Q_0 :a special state called initial state

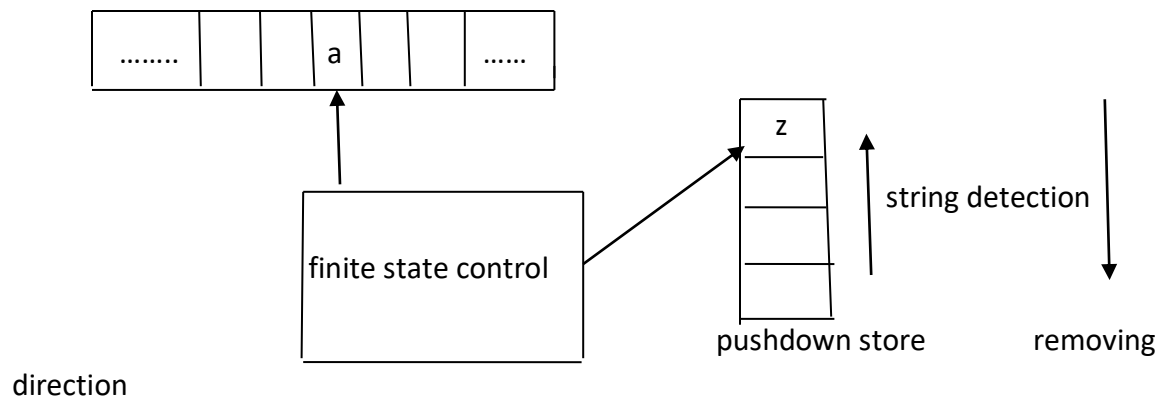
Z_0 :a special push down symbols called the initial symbols on the pushdown storedenoted by z_0 .

In otherwords indicates the top of the stack

F :the pushdown symbol denoting the final state.

COMPONENTS OF PUSH DOWN AUTOMATA:

The following diagram denotes the basic model of a pushdown automata



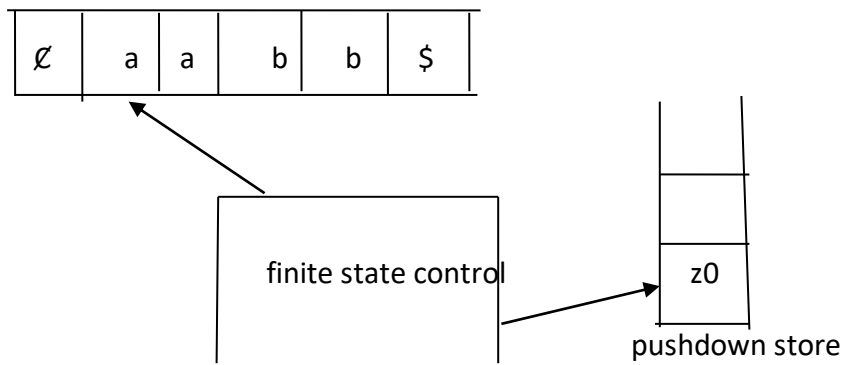
it has a read only input tape, input alphabet, finite state control, set of final states and an initial state as in the case of finite automata.

But in addition to this it has a stack called the pushdown store, it is a read write pushdown store as we add elements to the pds or remove elements from pds. a finite automata is in same state and on reading an input symbol the top most symbol in pds moves to a new state and writes a string of symbols in the pds.

DESIGN FOR PUSH DOWN AUTOMATA:

Consider the context free languages $a^n b^n, n \geq 1$, from this sample it can be concluded that the language consists of equal number of a's as equal number of b's. let us consider the case where $n=2$, then we

get the string aabb, the following figure shows the structure of pda for the above string.



there are three states to be considered in this case they are:

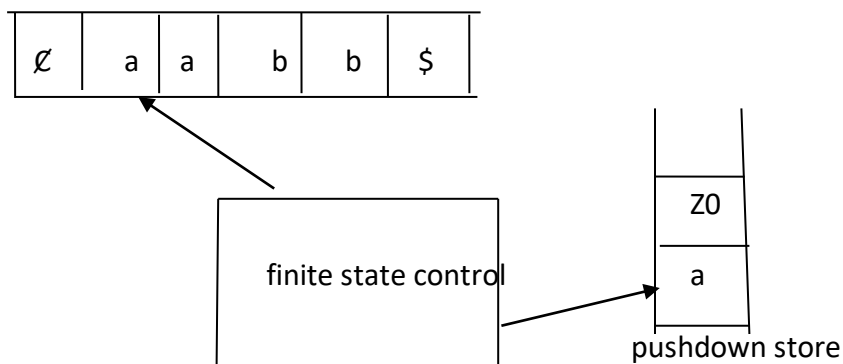
q_0 it is the state that recognizes the beginning alphabets(i.e)the symbol a

q_1 it recognizes the second half of the strings (i.e)the symbol b

q_2 it is the state that accepts all the alphabets now we start the design of the pushdown automata.

Step1:

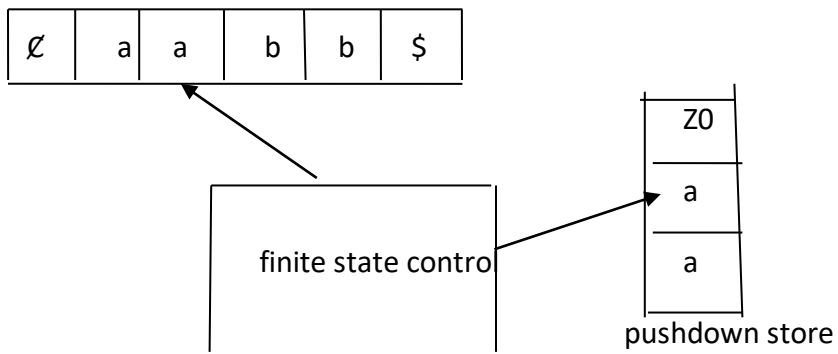
$\delta(q_0, a, z_0)$



in this case of the beginning alphabet a is pushed to the stack.and head pointer is incremented.

Step 2:

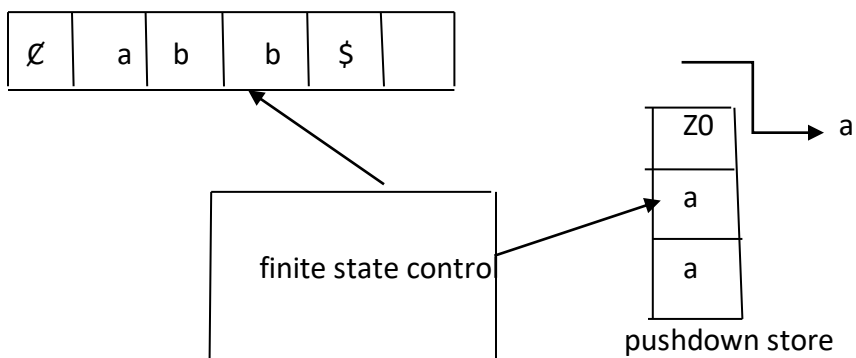
$\delta(q_0, a, az_0)$



the next alphabet is recognized by q_0 and pushed into the stack.

Step 3:

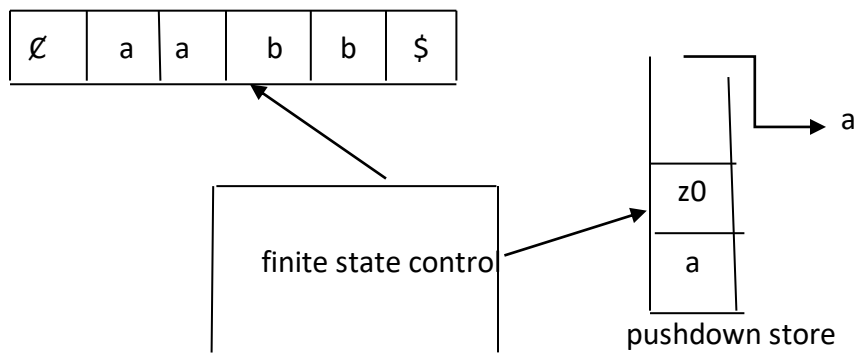
$\delta(q_1, b, aaz_0)$



in this case we find that we encounter a new alphabet b. upon this the alphabet a is popped from the stack.

Step 4:

$\delta(q_1, b, az_0)$



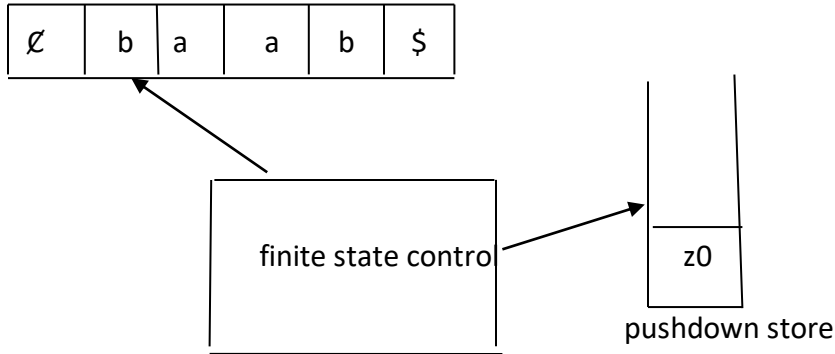
here again we encountered b, so another is popped from the stack.

this is the final state in which we again end up having the stack empty. because after all the a's in the given string are added to the stack. when the symbol b is encountered in the input string an a is removed from the stack bases. this confirms the statement equal number of a's and equal number of b's in the string.

An example for palindrome checking

In this case we consider a string baab and design a push down automata to determine if the given string is palindrome or not.

The following figure shows basic PDA structure for the string.



CONTEXT FREE GRAMMERS:

$P \rightarrow P$

$P \rightarrow 0$

$P \rightarrow 1$

$P \rightarrow 0P0$

$P \rightarrow 1P1$

A context free grammar for palindromes

There are four important components in a grammatical description of a language:

1. There is a finite set of symbols that form the strings of the language being defined. This set was $\{0, 1\}$ in the palindrome example we just saw. We call this alphabet the terminals, or terminal symbols.
2. There is a finite set of variables, also called sometimes nonterminals or syntactic categories. Each variable represents a language; i.e., a set of strings. In our examples above, there was only one variable, P , which we used to represent the class of palindromes over alphabet $\{0, 1\}$.
3. One of the variables represents the language being defined; it is called the start symbol. Other variables represent auxiliary classes of strings that are used to help define the language of the start symbol. In our examples, P , the only variable, is the start symbol.

4. There is a finite set of productions or rules that represents the recursive definition of a language. Each production consists of:

(a) A variable that is being (partially) defined by the production. This variable is often called the head of the production.

(B) The production symbol →

(c) A string of zero or more terminals and variables. This string, called the body of the production, represents one way to form strings in the language of the production, represents one way to form strings in the language of the variable of the head. In so doing, we leave terminals unchanged and substitute for each variable of the body any string that is known to be in the language of that variable.

The four components just described form a context free grammar, or just grammar, or CFG. We shall represent a CFG G by its four components, that is, $G = (V, T, P, S)$, where V is the set of variables the terminals the set of productions, and S the start symbol.

TOP DOWN PARSING:

A parser for grammar G is a program that takes as input a string w and produces as output either a parse tree for w . the method or the processes involved in converting a input string to a parse tree.

There are two types of parsers.

a) top down parser

b) bottom up parser

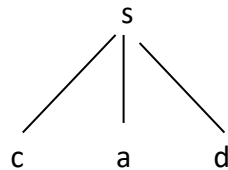
DIFFERENCE B\W TOPDOWN PARSING AND BOTTOM UP PARSING:

Bottom up parsers start with the bottom(leaves) to the top(root), while top down parsers start with the root and work down to the leaves. in both the cases the input to the parser is being scanned from left to right one symbol at a time.

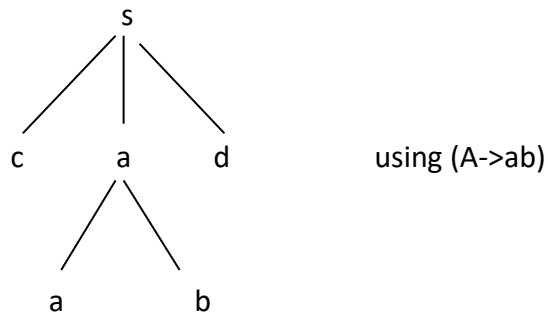
Top down parsing can be viewed as an attempt to find a left most derivation for an input string. equivalently, it can be viewed as attempting to construct a parse tree for the input starting from the root and creating the nodes of the parse tree in preorder.

For eg. $s \rightarrow cAd$ $A \rightarrow ab/a$ and the input $w=cad$.

To construct a parse tree for this sentence top-down, we initially create a tree consisting of a single node labeled s . an input pointer points in c , the first symbol to c , the first symbol of w . we then use first production for " s " to expand the tree and obtain

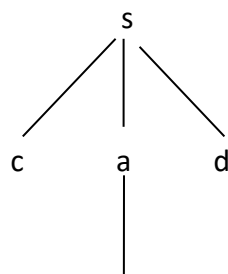


the left most leaf, labeled c , matches the first symbol of w , so we have advanced the input to a , the second symbol, of w and expand the first alternate for A to obtain the tree.



we now consider d , the third input symbol and next leaf, labeled b , since b does not match d , we report failure and go back to A to see whether there is another alternate for A that we have not tried but which produces the match.

In going back to A we must reset the input pointer to position q , the position it had when we first came to A . we now try the second alternate for A to obtain the tree.



d

the leaf matches the second symbol of w and the leaf d matches the third symbol. Since we have now produced a parse tree for w. We halt and announce successful completion of parsing.

An easy way to implement such a parser is to create a procedure for each non terminal

```
Procedure s()
Begin
If input symbol='c'then
Begin
ADVANCE()
If A ()then
Input symbol='d' then
Begin ADVANCE();return true end
End
Return false
End
Procedure A()
Begin
Isave=input-pointer;
If input symbol='a'then
Begin
ADVANCE();
If input symbol='a'then
Begin
ADVANCE();
If input symbol='b'then begin ADVANCE();
Return true end;
End
Input-pointer=Isave;
/*failure to find ab*\
If input -symbol='a'then
Begin ADVANCE();
Return true end
Else return false
End
```

The procedure ADVANCE() moves the input pointer to the next input symbol. "input symbol" is the one currently pointed to by the input pointer. procedure return true or false depending on whether or not they have found the non terminal. There are several difficulty with top down parsing. the first concerns left recursion.

A grammer G is said to be left recursive if it has a non terminal. a such that there is a derivation $A \rightarrow A^*$ for some $*$. so we should eliminate left recursion.

ELIMINATION OF LEFT RECURSION:

If we have the left recursive pair of production, $A \rightarrow A^* / \beta$ where β does not begin with on A, we can eliminate the left recursion by productions with

$$A \rightarrow \beta A'$$

$$A' \rightarrow *A' / \epsilon$$

For eg: consider the grammer

$$E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / id$$

Terminating the immediate left recursion ($A \rightarrow A^*$)

$$E \rightarrow TE'$$

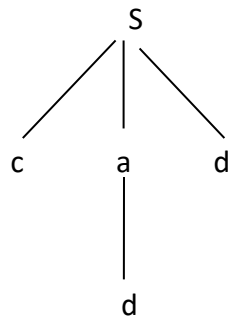
$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' / \epsilon$$

$$F \rightarrow (E) / id$$

The second problem with top down back tracking parsers is the order in which alternatives are tried can affect the language accepted. for eg, if we used 'a' and then ab as the order of the alternates for A. that is with parse trees and ca already matched the failure of the next input symbol 'b' to match



would imply that the alternate cAd for s was wrong ,leading to rejection of cabd.

For eg:

The top down parser generate the parse tree for the string 'abab'using grammer

$S \rightarrow AB/\epsilon$

$A \rightarrow a$

$B \rightarrow BS$

$B \rightarrow b$

$S \rightarrow AB$

$\rightarrow a B \quad (A \rightarrow a)$

$\rightarrow a B S \quad (B \rightarrow BS)$

$\rightarrow a b S \quad (B \rightarrow b)$

$\rightarrow abAB \quad (S \rightarrow AB)$

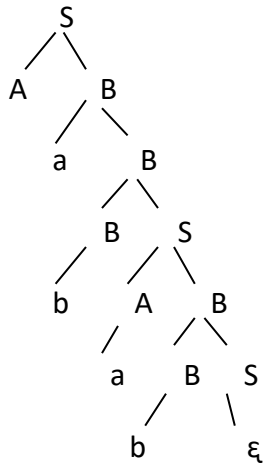
$\rightarrow abaB \quad (A \rightarrow a)$

$\rightarrow abaBS \quad (B \rightarrow BS)$

$\rightarrow ababs \quad (B \rightarrow b)$

$\rightarrow abab \quad (s \rightarrow \epsilon)$

parse tree(root)



BOTTOM UP PARSING:-

Bottom-up parses build parse trees from the bottom (leaves) to the top (root). The bottom up parsing method we discuss is called the “shift reduce” parsing because it consists of shifting input symbols onto a stack until the right side of a production appears on top of the stack. The right side may then be replaced by (reduced to) the symbol on the left of the production, and the process repeated.

Shift-Reduce parsing:-

In this section we discuss a bottom-up parsing called shift-reduce parsing. This parsing method is bottom-up parsing because it attempts to construct a parse tree for an input string beginning at the leaves(the bottom) and working up towards the root(the top). We can think of this process as one of “reducing” a string ‘w’ to the start symbol of a grammar. In each step a string matching the right side of a production is replaced by the symbol on the left.

For example consider the grammar given below:

$S \rightarrow aAcBc$

$A \rightarrow Ab/b$

$B \rightarrow d$

and the string to be derived is abbcde. We want to reduce this string S. we scan abbcde looking for substrings that match the right side of some production. The substrings b and d qualify. Let us choose the leftmost b replace it by A, the left side of the production $A \rightarrow b$. we obtain the string aAbcde. We now find that Ab, b and d each match the right side of some production.

Suppose this time we choose to replace the substrings Ab by A, the left side of the production $A \rightarrow Ab$. We obtain aAcde. Then replacing d by B, the left side of the production $B \rightarrow d$, we obtain aAdBc. We can now replace this entire string by S.

Each representation of the right side of a production by the left side in the process is called a reduction. Thus, by a sequence of four reductions we were able to reduce abbcde to S. these reductions. In fact, traced out a rightmost derivation in reverse.

Informally, a substring which is the right side of production such that replacement of that string by the production, process of bottom up parsing may be viewed as of finding and reducing handles.

We must not be misled by the simplicity of this example. In many cases the leftmost substring B which matches the right side of the function.

HANDLES:-

A handle of a right-sentential form γ is a production $A \rightarrow \beta$ a position of γ where the string β may be found and replaced by A to produce the previous right-sentential form in a right-most

derivation of γ . That is, if $S \rightarrow \alpha\beta w$, then $A \rightarrow \beta$ in the position following α is a handle of $\alpha\beta w$. the string w to the right of the handle contains only terminal symbols.

In the above example, $abbcde$ is a right-sentential form whose handle is $A \rightarrow b$ at position 2 likewise, $aAbcde$ is a right-sentential form whose handle is $A \rightarrow Ab$ at position 2.

Sometimes we shall say, "The substring β is a handle of $\alpha\beta w$ ", if the position of B and the production $A \rightarrow \beta$. We have in mind are clear. If a grammar is ambiguous, then every right-sentential form, so the grammar has exactly one handle.

The below diagram portrays the handle β in the parse tree of the right-sentential form $\alpha\beta w$ the handle represents the leftmost complete sub tree consisting of the node with its children in the tree.

For example consider the grammar:-

$$E \rightarrow E+E ; E \rightarrow (E)$$

$$E \rightarrow E^*E ; E \rightarrow id$$

And consider the rightmost derivation

$$E \rightarrow E+E$$

$$\rightarrow E+E^*E$$

$$\rightarrow E+E^*id_3$$

$$\rightarrow E+id_2^*id_3$$

$$\rightarrow id_1+id_2^*id_3$$

For example id_1 is a handle of the right sentential form $id_1+id_2+id_3$ because id is the right most side of the production $E \rightarrow id$, and replacing id_1 by E produces the previous right-sentential form $E+id_2^*id_3$. Note that the string appearing to the right of a handle contains only terminals symbols.

HANDLE PRUNING:-

A rightmost derivation in reverse, often called a canonical reduction sequence, is obtained by “handle pruning”. That is we start with a string of terminals ‘w’ which we wish to parse. If ‘w’ is a sentence of the grammar at hand, then $w = \gamma_n$ is the nth right-sentential form of some as yet unknown rightmost derivation.

$$S = \gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \dots \rightarrow \gamma_{n-1} \rightarrow \gamma_n = w$$

To reconstruct this derivation, in reverse order. We locate the handle β_n in γ_n and replace β_n by the left side of some derivations. $A_n \rightarrow B_n$ to obtain the (n-1)st right-sentential form γ_{n-1} and then repeat this process that is we locate the handle β_{n-1} in γ_{n-1} and reduce this handle to obtain the right sentential form γ_{n-2} if by considering this process we produce a right sentential form considering only of the start symbol(S).

Consider the example given below :

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E)$$

$$E \rightarrow id$$

Input string is id1+id2*id3

Right-sentential form	Handle	Reducing production
id1+id2*id3	id1	$E \rightarrow \text{id}$
$E+\text{id2*id3}$	id2	$E \rightarrow \text{id}$
$E+E*\text{id3}$	id3	$E \rightarrow \text{id}$

We can observe that the sequence of right-sentential forms in this example is just the reverse of the sequence in the right-most derivation.

STACK IMPLEMENTATION OF SHIFT REDUCE PARSING:-

To implement a shift-reduce parser is to use a stack and a input buffer. We shall use \$ to mark the bottom of the stack and the right end of the input.

Stack	input
\$ S	\$

In this configuration the parser halts and announces successful completion.

Consider the given example:

Stack	Input buffer	Action
\$	id1+id2*id3\$	shift
\$id1	+id2*id3\$	reduce id1
\$E	+id2*id3\$	shift
\$E+	id2*id3\$	shift
\$E+id2	*id3\$	reduce id2
\$E+E	*id3\$	shift

y four

- Shift
- Reduce
- Accept
- Error

- a) In a shift action, the next input symbol is shifted to the top of the stack.
- b) In a reduce action, the parser knows the right end of the handle is at the top of the stack. It must locate the left end of the handle within the stack and decide with what non-terminal to replace the handle.
- c) In an accept action, the parser announces successful completion of parsing.
- d) In an error action, the parser discovers that a syntax error has occurred and calls an error recovery routine.

APPLICATION OF PUMPING LEMA:

APPLICATION:

1. check whether the given language is context free language or not.
2. to check whether the string is finite or infinite.
3. Pumping level for cfl's states that there are always two short substrings close together that can be repeated, both the same number of times as often we like.

STATEMENT:

Let "L" be any CFL (context free language). then there is a constant n, depending only on L, such that "z" is in L and $|z| \geq n$, then we may write $z = uvwxy$ such that,

a) $|vx| \geq 1$

b) $|vwx| \leq n$ and

for all $i > 0$, uv^iwx^iy is in L.

TO PROVE:

To check whether uv^iwx^iy is in context free language "L" or not

PROOF:

Let G be a Chomsky normal form grammar generating L(G). observe that if Z is in L(G) and Z is long, then any parse tree for x must contain a long path. more precisely we can show by induction on "i" that if the parse tree of a word generated by a Chomsky normal form grammar has no path of length greater than i, then the word is of length no greater than $2^i - 1$.

BASIS:

For the path $i=1$, the derivation tree is of the form,



here $n \leq 2^i - 1$ where $n=1, i=1$

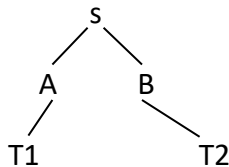
$$l \leq 2^i - 1$$

$$l \leq 1$$

hence basis is proved.

INDUCTION PART:

For this consider $i > 1$. let the roots are



If there are no path greater than l , the words is of length no greater than $2^i - 1$. if there are no greater than $2^i - 1$. if there are no paths greater than $2^i - 1$, the word is of length no greater than $2^i - 2$ or fewer symbols.

For proof was the Chomsky normal form grammer,

$$G = (\{A, B, C\}, \{a, b\}, A \rightarrow BC, C \rightarrow BA, A \rightarrow a, B \rightarrow b, A)$$

Where,

$$V = \{A, B, C\} \quad T = \{a, b\}$$

$$P = \{A \rightarrow BC, B \rightarrow BA, C \rightarrow BA, B \rightarrow b\}$$

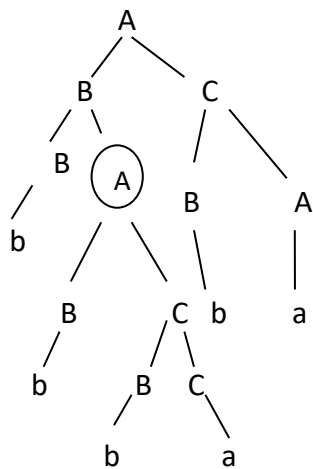
$$S = \{A\}$$

Consider a string $Z = bbbaba$ in this grammer. the derivation and derivation tree is as follows.

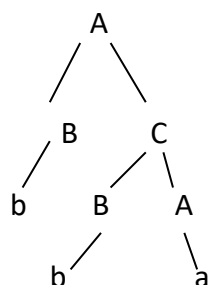
$$A \rightarrow BC$$

->BAC (replace B->BA)
 ->b AC (replace B->b)
 ->b BCC (replace A->BC)
 ->bb CC (replace B->b)
 ->bbBAC (replace C->BA)
 ->bbbAC (replace B->b)
 ->bbbaC (replace A->C)
 ->bbbaBA (replace C->BA)
 ->bbbabA (replace B->b)
 ->bbbaba (replace A->a)

DERIVATION TREE:



a) subtree with parent vertex(a)v1:



subtree with parent vertex(a)v2:

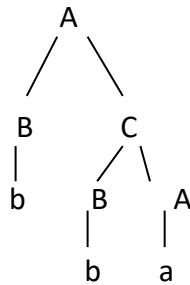


in the parse tree, let p be a path that is long or longer than any path in the tree. then there must be two vertices v_1 and v_2 on the path satisfying the following conditions,

- a) the vertices v_1 and v_2 both have same label way A .
- b) vertex v_1 is closer to the root than vertex v_2 .
- c) the position of the path v_1 to the leaf is of length almost $k+1$.

To use that V_1 and V_2 can be always found, just proceed up path p from the leaf, keeping track of the labels encountered. of the first $k+2$ vertices only the leaf has a terminal label.

Consider a substring "bba" from the string bbbaba containing the path length $i=3$



path $i=3$

thus,

$$A \Rightarrow bba = w$$

Since path length $i=3$

$$|w| \leq 2^{i-1} \quad |w| = 3$$

$$3 \leq 2^3 - 1$$

$$3 \leq 2^2$$

$$3 \leq 4$$

Which holds the condition $|w| \leq 2^{i-1}$ for pumping lemma for cfl.

CLOSURE PROPERTIES OF CFL:

The closure properties of CFL are

1. CFL's are closed under union, concatenation and Kleene closure.
2. CFL's are closed under homomorphism
3. CFL's are closed under substitutions
4. If L is a CFL and R is a regular language then, L/R is a CFL.

5.CFL's are not closed under intersection

6.CFL's are closed under complementation.

Theorem 6.1 Context-free languages are closed under union, concatenation and kleene closure.

Proof : let L_1 and L_2 be CFL's generated by the CFG's

$$G_1 = \{V_1, T_1, P_1, S_1\} \text{ and } G_2 = \{V_2, T_2, P_2, S_2\}$$

Respectively. since we may rename variables at will without changing the language generated, we assume that V_1 and V_2 are disjoint. Assume also that S_2, S_4 and S_5 are not in V_1 or V_2 .

For $L_1 \cup L_2$ construct grammar $G_3 = (V_1 \cup V_2 \cup \{S_3\}, T_1 \cup T_2, P_3, S_3)$, where P_3 is $P_1 \cup P_2$ plus the production $S_3 \rightarrow S_1 \mid S_2$. If w is in L_1 , then the derivation $S_3 \Rightarrow S_1 \Rightarrow w$ is a derivation in G_3 , as every production of G_1 is a production of G_3 . similarly, every word in L_2 has a derivation in G_3 beginning with $S_3 \Rightarrow S_2$. Thus $L_1 \cup L_2 \subseteq L(G_3)$. For the converse, let w be in $L(G_3)$. Then the derivation $S_3 \Rightarrow w$ begins with either $S_3 \Rightarrow S_1 \Rightarrow w$ or $S_3 \Rightarrow S_2 \Rightarrow w$. In the former case, as V_1 and V_2 are disjoint, only symbols of G_1 may appear in the derivations $S_1 \Rightarrow w$. As the only productions of P_3 that involve only symbols of G_1 are those from P_1 , we conclude that only productions of P_1 are used in the derivations $S_1 \Rightarrow w$. Thus $S_1 \Rightarrow w$, and w is in L_1 . Analogously, if the derivation starts $S_3 \Rightarrow S_2$, we may conclude w is in L_2 . Hence $L(G_3) \subseteq L_1 \cup L_2$, so $L(G_3) = L_1 \cup L_2$ as desired.

For concatenation, let $G_4 = (V_1 \cup V_2 \cup \{S_4\}, T_1 \cup T_2, P_4, S_4)$, where P_4 is $P_1 \cup P_2$ plus the production $S_4 \rightarrow S_1 S_2$. A proof that $L(G_4) = L(G_1)L(G_2)$ is similar to the proof for union and is omitted.

For closure, let $G_5 = (V_1 \cup V_2 \cup \{S_5\}, T_1 \cup T_2, P_5, S_5)$, where P_5 is P_1 plus the productions $S_5 \rightarrow S_1 S_5 \mid \epsilon$. We again leave the proof that $L(G_5) = L(G_1)^*$ to the reader.

SUBSTITUTION AND HOMOMORPHISMS

Theorem 6.2 The context-free languages are closed under substitution.

Proof : Let L be a CFL, $L \subseteq \Sigma^*$, and for each a in Σ let L_a be a CFL. Let L be $L(G)$ and for each a in Σ let L_a be $L(G_a)$. Without loss of generality assume that the variables of G and the G_a 's are disjoint. Construct a grammar G' as follows the variables of G' are all the variables of G and the G_a 's, the terminals of G are the terminals of the G_a 's. the start symbol of G' is the start symbol of G . The productions of G' are all the productions of the G_a 's together with those productions formed by taking a production $A \rightarrow \alpha$ of G and substituting S_a , the start symbol of G_a , for each instance of an a in Σ appearing in α .

Example 6.4 let L be the set of words with an equal number of a 's and b 's $L_a = \{0^n 1^n \mid n \geq 1\}$ and $L = \{ww^r \mid w \text{ is in } (0+2)^*\}$. For G we may choose

$$S \rightarrow asbs \mid bsas \mid \epsilon$$

For G_a take $S_a \rightarrow 0S_a1 \mid 01$

For G_b take $S_b \rightarrow 0S_b0 \mid 2S_b2 \mid \epsilon$

If f is the substitution $f(a) = L_a$ and $f(b) = L_b$, then $f(L)$ is generated by the grammar

$$S \rightarrow Sa SSbS \mid Sb SSa S \mid \epsilon$$

$$S_a \rightarrow 0S_a1 \mid 01$$

$$S_b \rightarrow 0S_b0 \mid 2S_b2 \mid \epsilon$$

One should observe that since $\{a,b\}, \{ab\}$, and CFL's the closure of CFL's under substitute implies closure under union, concatenation, and $*$. The union of L_a and L_b is simply the substitute of L_a and L_b into $\{a,b\}$ and similarly $L_a L_b$ and L_a^* are tge substitute into $\{ab\}$ and a^* , respectively. Thus theorem 6.1 could be presented as a corollary of theorem 6.2.

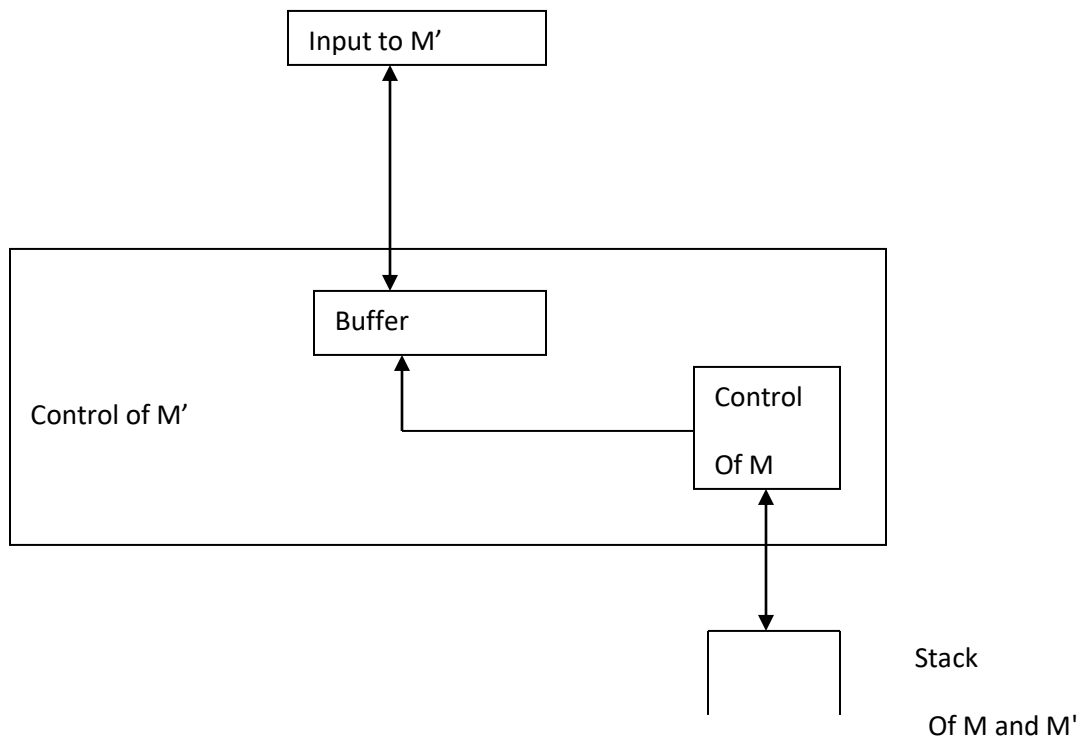
Since a homomorphism is a special type of substitution we state the following corollary.

Corollary The CFL's are closed under homomorphism.

Theorem 6.3 The context-free languages are closed under inverse homomorphism .

Proof : As with regular sets, a machine-based proof for closure under inverse homomorphism is easiest to understand. Let $h: \Sigma \rightarrow \Delta$ be a homomorphism and L be a CFL. Let $L = L(M)$, where M is the PDA $(Q, \Delta, \Gamma, \delta, q_0, Z_0, F)$. In analogy with the finite-automaton construction of theorem 3.5, we construct PDA M' accepting $h^{-1}(L)$ as follows. On input w , M' generates the string $h(w)$ and simulates M on $h(w)$. If M' were a finite automaton, all it could do on a string $h(w)$ would be to change state, so M' could simulate such a composite move in one of its moves. However, in the PDA case, M could pop many symbols on a string, or, since it is nondeterministic, make moves that push an arbitrary string. To simulate M 's moves on $h(w)$ with one (or any finite number of) moves of its own.

What we do is give M' a buffer, in which it may store $h(w)$. Then M' may simulate any ϵ -moves of M it likes and consume the symbols of $h(w)$ one at a time, as if they were M 's input. As the buffer is a part of M' 's finite control, it cannot be allowed to grow arbitrarily long. We ensure that it does not, by permitting M' to read an input symbol only when the buffer is empty. Thus the buffer holds a suffix of $h(w)$ for some a at all times. M' accepts its input w if the buffer is empty and M is in a final state. That is, M has accepted $h(w)$. thus $L(M') = \{w \mid h(w) \text{ is in } L\}$, that is



Construction of a PDA accepting $h^{-1}(L)$

$L(M') = h^{-1}(L(M))$. The arrangement is depicted in fig ; the formal construction follows.

Let $M' = (Q', \Sigma, \Gamma, \delta', [q_0, \epsilon], Z_0, F \times \{\epsilon\})$, where Q' consists of pair $[q, x]$ such that q is in Q and x is a (not necessarily proper) suffix of some $h(a)$ for a in Σ . δ' is defined as follows:

- 1] $\delta'([q, x], \epsilon, Y)$ contains all $([p, x], y)$ such that $\delta(q, \epsilon, Y)$ contains (p, y) . simulate ϵ -moves of M independent of the buffer contents.
- 2] $\delta'([q, ax], \epsilon, Y)$ contains all $([p, x], y)$ such that $\delta(q, a, Y)$ contains (p, y) . Simulate moves of M on input a in Δ , removing a from the front of the buffer.
- 3] $\delta'([q, \epsilon], a, Y)$ contains $(q, h(a)), y$ for all a in Σ and Y in Γ . Load the buffer with $h(a)$, reading a from M' 's input ; the state of M remain unchanged.

To show that $L(M') = h^{-1}(L(M))$ first observe that by one application of rule (3), followed by application of rules (1) and (2), if $(q, h(a), \alpha)^* / M(p, \epsilon, \beta)$.

Theorem 6.4: The CFL'S are not closed under intersection.

Proof: we showed the language $L_1 = \{a^i b^j c^k \mid i \geq 1\}$ was not a CFL. We claim that $L_2 = \{a^i b^j c^k \mid i \geq 1 \text{ and } j \geq 1\}$ and $L_3 = \{a^i b^j c^k \mid i \geq 1 \text{ and } j \geq 1\}$ are both CFL's. For exa, [;e. a {DA to recognize L_2 stores the a 's pm its stack and cancels them against b 's, then accepts its input after seeing one or mors c 's. Alternatively L_2 is generated by the grammar

$S \rightarrow AB$

$A \rightarrow aAb \mid ab$

$B \rightarrow cB \mid c$

Where A generates $a^i b^j$ and B generates c^k . A similar grammar

$S \rightarrow CD$

$C \rightarrow aC \mid a$

$D \rightarrow bDc \mid bc$

Generates L_3 .

However, $L_2 \cap L_3 = L_1$.

If the CFL's were closed under intersection. L_1 would thus be a CFL.

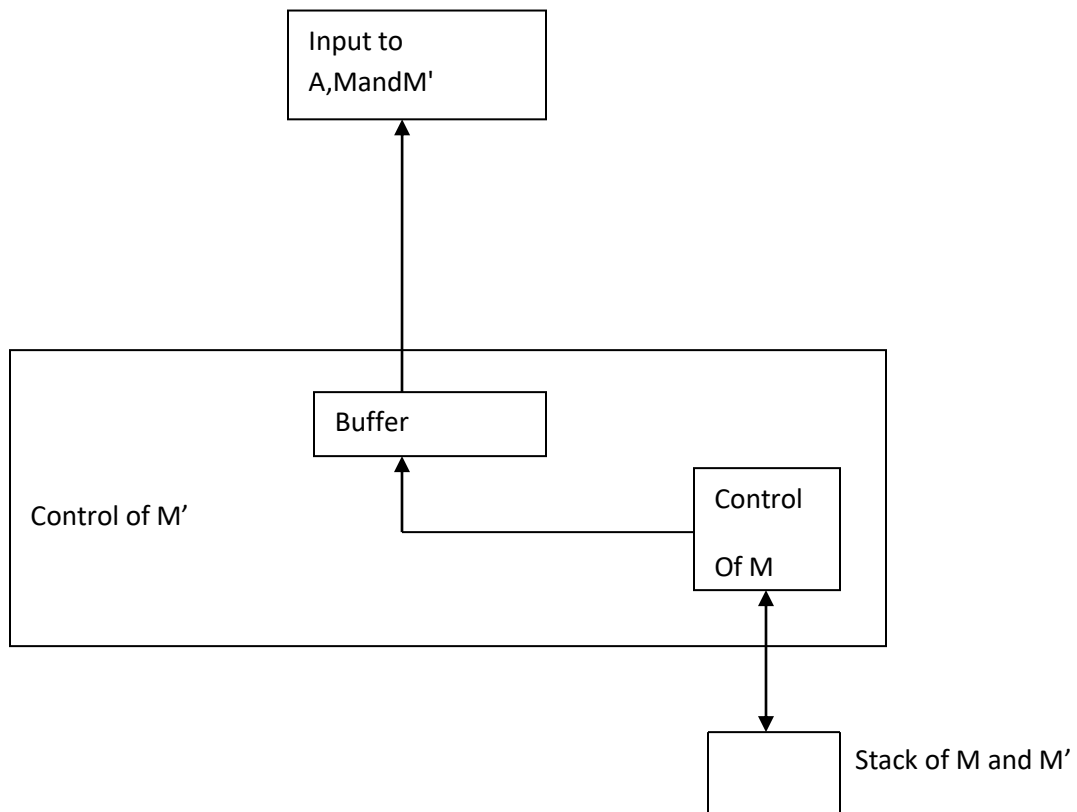
Corollary : The CFL's are not closed under complementation.

Proof: we know the CFL's are closed under union. If they were closed under complementation, they would, by DeMorgan's law, $L_1 \cap L_2 = L_1 \cup L_2$, be closed under intersection.

Theorem 6.5: If L is a CFL and R is a regular set, then $L \cap R$ is a CFL.

Proof: Let L be $L(M)$ for PDA $M = (Q_m, \Sigma, \Gamma, \delta, [p_0, q_0], z_0, f_a \times f_m)$, and let R be $L(A)$ for DFA $A = (Q_a, \Sigma, \delta_a, p_0, f_a)$. We construct a PDA M' for $L \cap R$ by "running M and A in parallel, M' simulates moves of M on input without changing the state of A . when M' makes a move on input symbol a , M simulates that move and also simulates A 's change of state on input a . M' accepts if and only if both A and M accept. Formally, let

$$M' = (Q_a \times Q_m, \Sigma, \Gamma, \delta', [p_0, q_0], z_0, F_a \times F_m)$$



DECISION ALGORITHM FOR CFL:

STATEMENT:

Decision algorithm for cfl is mainly used to check whether the given language is finite, empty or non finite (infinite).

TO PROVE:

To check whether the language is finite, infinite or empty.

ASSUMPTIONS:

1. the grammar should be in Chomsky normal form (cnf) without ϵ production.

2. if 's' is the start symbol and 'r' is the rank of the symbol, then the string length will be greater than or equal to 2^r .

If 'A' is the reverse vertex associated with 's' is the root node, then the string length will not be greater than 2^{r-1} .

If 'B' is the reverse vertex associated with 's' is the root node, then the string length will not be greater

than 2^{r-2} .

BASIS PART: (r=0)

Consider a production $A \rightarrow a$, it is a directed acyclic graph denoted by . by the assumption a non terminal with rank 'r' cannot generate a string of length greater than 2^r .

Here, the rank of A is '0'. i.e. (r=0)

By the condition $l \leq 2^r$, substitute 'r' value,

We get $l \leq 2^0$

$$l \leq 1$$

Hence it is proved by considering the form $A \rightarrow a$, we may derive only a string of length '1', which is finite.

INDUCTION PART:

If we use a production of the form $A \rightarrow a$, we may derive only a string of length 1. if we begin with $A \rightarrow BC$, then as B and C are of rank $r-1$ or less, by the inductive hypothesis, they derive only strings of length 2^{r-1} or less. thus BC cannot derive a string of length greater than 2^r .

Since s is of finite rank r_0 , and in fact, is of rank no greater than the number of variables, s derives strings of length no greater than the number of variables, s derives strings of length no greater than 2^{r_0} . thus the language is finite.

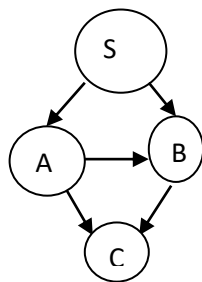
Consider an example,

$S \rightarrow AB$

$A \rightarrow BC$

$B \rightarrow CC$

$C \rightarrow a$



$$r(s)=3$$

$$r(A)=2$$

$$r(B)=1$$

$$r(c)=0$$

rank of starting symbol 's' ($r(s)=3$).

Rank 'A', $r(A)$ depends on root node 's', it should be less than 's' and rank of 'b', $r(B)$ depends on both S and A, it should be less than both S and A.

From, the figure check the condition $l \leq 2^r$

$$r(s)=3, \quad l \leq 2^3$$

$$l \leq 8$$

$$r(A)=2, \quad l \leq 2^2$$

$$l \leq 4$$

$$r(B)=1, \quad l \leq 2^1$$

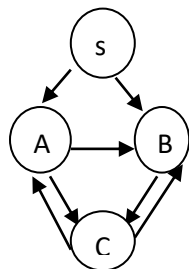
$$l \leq 2$$

$$r(c)=0, \quad l \leq 2^0$$

$$l \leq 1$$

hence the finite length for each string is derived ..hence it proved.

Now, consider the directed cyclic graph,



For the figure, the rank cannot be specified. hence for the cyclic graph the language is infinite.

For cyclic graph the language is finite. hence the statement is proved.